

Introduction to Databases – SQL Querying Training

Practice exercise, same style and difficulty as the example exam

Exercise: Querying

Consider the following relational schema, keeping track of conference participants, papers they want to read, and ratings they give after reading.

PARTICIPANT (Pid, Name, YoB)	The participant identified by PId has name Name and was born in year YoB.
PAPER (DOI, Title, Author, Year)	The paper identified by DOI has title Title, main author Author, and publication year Year.
READLIST (Pid, DOI)	Participant PId has added the paper DOI to their reading list.
EVALUATION (Pid, DOI, Score)	Participant PId evaluated paper DOI with score Score, from 1 to 5. Only one evaluation per participant and paper is allowed.

Assume the following usual integrity constraints: READLIST[PId] subset PARTICIPANT[PId], READLIST[DOI] subset PAPER[DOI], EVALUATION[PId] subset PARTICIPANT[PId], and EVALUATION[DOI] subset PAPER[DOI].

Write the following queries in SQL syntax, and, where possible, in relational algebra. Use as few subqueries as possible, and clearly mark when DISTINCT is needed.

a)	The paper titles and authors in the reading list of the participant named Alice.
b)	The papers, by DOI, that have been added to at least one reading list but have never been evaluated.
c)	The papers, by title and author, that have been evaluated by more than one participant.
d)	The name or names of authors having the highest average evaluation score.
e)	The name or names of authors for which none of their papers has ever been evaluated.
f)	List the number of distinct authors for each publication year, for years in which at least one paper was published.

Solutions

a) Paper titles and authors in Alice's reading list

SQL:

```
SELECT DISTINCT p.Title, p.Author
FROM PARTICIPANT pa
JOIN READLIST r ON r.PId = pa.PId
JOIN PAPER p ON p.DOI = r.DOI
WHERE pa.Name = 'Alice';
```

DISTINCT is needed because Name is not declared unique, so there may be several participants named Alice, and because different DOI values can project to the same (Title, Author) pair.

Relational algebra:

$(\text{PARTICIPANT:Name='Alice'} * \text{READLIST} * \text{PAPER})[\text{Title, Author}]$

With theta joins, explicitly use PARTICIPANT.PId = READLIST.PId and READLIST.DOI = PAPER.DOI.

b) Papers in a reading list but never evaluated

SQL:

```
SELECT DOI
FROM READLIST
EXCEPT
```

```
SELECT DOI
FROM EVALUATION;
```

Use MINUS instead of EXCEPT in Oracle-style SQL. DISTINCT is not needed because EXCEPT/MINUS removes duplicates.

Alternative SQL:

```
SELECT DISTINCT r.DOI
FROM READLIST r
WHERE NOT EXISTS (
  SELECT 1
  FROM EVALUATION e
  WHERE e.DOI = r.DOI
);
```

Relational algebra:

READLIST[DOI] - EVALUATION[DOI]

c) Papers evaluated by more than one participant

SQL, self-join version:

```
SELECT DISTINCT p.Title, p.Author
FROM PAPER p
JOIN EVALUATION e1 ON e1.DOI = p.DOI
JOIN EVALUATION e2 ON e2.DOI = e1.DOI
AND e1.PId < e2.PId;
```

$e1.PId < e2.PId$ ensures that two different participants exist and avoids symmetric pairs. DISTINCT is still needed because a paper with many evaluations creates several pairs, and different papers may have the same title and author.

SQL, grouped version:

```
WITH MoreThanOnce AS (
  SELECT DOI
  FROM EVALUATION
  GROUP BY DOI
  HAVING COUNT(*) > 1
)
SELECT DISTINCT p.Title, p.Author
FROM PAPER p
JOIN MoreThanOnce m ON m.DOI = p.DOI;
```

Relational algebra:

```
E1(p1, doi1, score1) <- EVALUATION[PId, DOI, Score]
E2(p2, doi2, score2) <- EVALUATION[PId, DOI, Score]
ManyDOI(doi) <- (E1(E1.doi1 = E2.doi2 and E1.p1 <> E2.p2)*E2)[doi1]
Result <- (ManyDOI * PAPER)[Title, Author]
```

d) Authors with the highest average evaluation score

SQL:

```
WITH AuthorAverage AS (
  SELECT p.Author, AVG(e.Score) AS avg_score
  FROM PAPER p
  JOIN EVALUATION e ON e.DOI = p.DOI
  GROUP BY p.Author
), MaxAverage AS (
  SELECT MAX(avg_score) AS max_avg_score
  FROM AuthorAverage
)
SELECT Author
```

```
FROM AuthorAverage
JOIN MaxAverage ON avg_score = max_avg_score;
```

DISTINCT is not needed because AuthorAverage has one row per author. This preserves ties. Basic relational algebra from the course slides cannot express this query because it needs AVG, GROUP BY, and MAX.

e) Authors with no evaluated papers

SQL:

```
SELECT Author
FROM PAPER
EXCEPT
SELECT p.Author
FROM PAPER p
JOIN EVALUATION e ON e.DOI = p.DOI;
```

Use MINUS instead of EXCEPT in Oracle-style SQL. DISTINCT is not needed because the set operator removes duplicates.

Alternative SQL:

```
SELECT DISTINCT p.Author
FROM PAPER p
WHERE NOT EXISTS (
    SELECT 1
    FROM PAPER p2
    JOIN EVALUATION e ON e.DOI = p2.DOI
    WHERE p2.Author = p.Author
);
```

Relational algebra:

$PAPER[Author] - (PAPER * EVALUATION)[Author]$

f) Number of distinct authors for each publication year

SQL:

```
SELECT Year, COUNT(DISTINCT Author) AS author_count
FROM PAPER
GROUP BY Year;
```

COUNT(DISTINCT Author) is needed because the same author may have several papers in the same year. No HAVING clause is needed because grouping over PAPER already returns only years in which at least one paper exists. Basic relational algebra from the course slides cannot express this query because it needs grouping and counting.

End of training exercise.